

Tema 5

Introducción a SQL. Consultas

1. [Introducción](#)
2. [Elementos del lenguaje SQL](#)
3. [Sintaxis utilizada](#)
4. [Tipos de datos](#)
5. [Consulta de datos: SELECT](#)
 - 5.1. [Cláusula SELECT](#)
 - 5.2. [Cláusula FROM](#)
 - 5.3. [Cláusula WHERE](#)
 - 5.4. [Cláusula ORDER BY](#)
 - 5.5. [Cláusula LIMIT](#)
6. [Combinación de tablas \(consultas multitabla\)](#)
7. [Subconsultas](#)
8. [Anexos](#)
 - 8.1. [Ejemplos de combinación de tablas](#)
 - 8.2. [Diccionario de datos](#)
 - 8.3. [Tablas de los ejercicios](#)

1. Introducción

Se trata de uno de los lenguajes más utilizados de la historia de la informática. Es, sin duda, el lenguaje fundamental para manejar una base de datos relacional y, por tanto, es el lenguaje fundamental de los SGBD relacionales.

SQL es un lenguaje declarativo en lo que lo importante es definir qué se desea hacer, por encima de cómo hacerlo (que es la forma de trabajar de los lenguajes de programación de aplicaciones como C o Java). Con este lenguaje se pretendía que las instrucciones se pudieran escribir como si fueran órdenes humanas; es decir, utilizar un lenguaje lo más natural posible. De ahí que se le considere un lenguaje de cuarta generación.

Se trata de un lenguaje que intenta agrupar todas las funciones que se le pueden pedir a una base de datos, por lo que es el lenguaje utilizado tanto por administradores como por programadores o incluso usuarios avanzados.

El nacimiento del lenguaje SQL data de 1970 cuando E. F. Codd publica su libro: *"Un modelo de datos relacional para grandes bancos de datos compartidos"*. Ese libro dictaría las directrices de las bases de datos relacionales. Apenas dos años después IBM (para quien trabajaba Codd) utilizó las directrices de Codd para crear el *Standard English Query Language* (Lenguaje Estándar Inglés para Consultas) al que se le llamó SEQUEL. Más adelante se le asignaron las siglas SQL (*Standard Query Language*, lenguaje estándar de consulta) aunque en inglés se siguen pronunciando *secuel*.

En 1979 Oracle presenta la primera implementación comercial del lenguaje. Poco después se convertía en un estándar en el mundo de las bases de datos avalado por los organismos ISO y ANSI. En el año 1986 se toma como lenguaje estándar por ANSI de los SGBD relacionales. Un año después lo adopta ISO, lo que convierte a SQL en estándar mundial como lenguaje de bases de datos relacionales.

En 1989 aparece el estándar ISO (y ANSI) llamado SQL89 o SQL1. En 1992 aparece la nueva versión estándar de SQL (a día de hoy sigue siendo la más conocida) llamada SQL92. En 1999 se aprueba un nuevo SQL estándar que incorpora mejoras que incluyen triggers, procedimientos, funciones... y otras características de las bases de datos objeto-relacionales; dicho estándar se conoce como SQL99. Desde entonces, se han sucedido diferentes versiones (2003, 2006, 2008, 2011) hasta llegar a la última, del año 2016 (SQL2016).

Aunque estos apuntes sirven como guía de uso de SQL estándar, el Sistema Gestor de Bases de Datos que se utilizará como referencia fundamental es MariaDB, que surgió como una derivación de MySQL con licencia GPL (MariaDB se consolidó como sustituto 1:1 con compatibilidad binaria para MySQL. Sin embargo, esta compatibilidad se interrumpe a partir de la versión MySQL 8.0). Existen otros muchos SGBD muy utilizados en la actualidad, entre los que destacan, el propio MySQL, Oracle, Microsoft SQL Server, PostgreSQL, SQLite... Además, existen herramientas que facilitan el trabajo directo con las bases de datos, como pueden ser MySQL Workbench, HeidiSQL, DBeaver, dbForge, phpMyAdmin, SQL Developer, DB Browser...

2. Elementos del lenguaje SQL

El código SQL consta de los siguientes elementos:

- **Comandos.** Los distintos tipos de instrucciones que se pueden realizar desde SQL son:
 - **DDL, Data Definition Language** (Lenguaje de definición de datos). Permiten crear, modificar y eliminar la estructura de las tablas de la base de datos. Lo forman las instrucciones **CREATE, ALTER, DROP, RENAME y TRUNCATE**.
 - **DML, Data Manipulation Language** (Lenguaje de manipulación de datos). Agrupa los comandos para manipular los datos contenidos en la base de datos. Lo forman las instrucciones **INSERT, UPDATE y DELETE**. Administran las modificaciones creadas por las instrucciones DML.
 - Dentro de este tipo, hay que destacar también la realización de **consultas mediante la orden SELECT**. A esta parte del lenguaje se le llama **DQL (Data Query Language, Lenguaje de consulta de datos)**; pero es parte del DML.
 - **DCL, Data Control Language** (Lenguaje de control de datos). Agrupa los comandos para administrar la seguridad de acceso a los datos. Las instrucciones más importantes de este grupo son **GRANT y REVOKE**.
 - **TCL, Transaction Control Language** (Lenguaje de control de transacciones), que agrupa los comandos utilizados para administrar la confirmación o no de actualizaciones realizadas sobre la base de datos. Las instrucciones más importantes de este apartado son **ROLLBACK y COMMIT**.
- **Cláusulas.** Son palabras especiales que permiten modificar el funcionamiento de un comando (WHERE, ORDER BY...).
- **Operadores.** Permiten crear expresiones complejas. Pueden ser aritméticos (+, -, *, /, ...), lógicos (>, <, !=, <>, AND, OR, ...).
- **Funciones.** Para conseguir valores complejos (SUM(), DATE(), ...).
- **Literales.** Valores concretos, valores fijos: números, textos, caracteres....
 - **Valores numéricos.** Se escriben con punto para separar los decimales. Ej: 2, 12.34, 186.366, etc.
 - **Valores de texto.** Se escriben entre comillas simples. Ej: 'Avda Cardenal Cisneros', 'España', '1236BMZ'
 - **Valores lógicos:** TRUE, FALSE
 - **Otros:** NULL (ausencia de valor; es diferente de 0 y diferente de una cadena vacía).
- **Metadatos.** Tablas que contiene información sobre la propia base de datos. Es importante conocer su estructura para averiguar información sobre la base de datos, sobre todo cuando no se dispone de una aplicación que facilite esa información.

A la hora de trabajar con MariaDB, también merece la pena destacar la configuración y **uso de las variables y los modos:**

- **Variables de usuario.** Se permite la utilización de variables de usuario específicas de la conexión mediante la sintaxis *@nombre_variable*. Un nombre de variable puede contener caracteres alfanuméricos, así como los signos `_`, `$` y `..`. No es necesario inicializar las variables antes de utilizarlas (de manera predeterminada tienen el valor NULL). Una variable de usuario puede contener un número (entero o real) o una cadena de caracteres. Se pueden utilizar en expresiones. Para asignar un valor a una variable, se puede utilizar el comando SET.

Ejemplo: SET @tipo_iva=21.0;

- **Variables de sistema.** Existen gran número de variables que permiten obtener información sobre el funcionamiento del servidor. En la mayoría de los casos, el valor de estas variables puede modificarse. Hay 2 tipos de variables de sistema: globales (afectan al conjunto del servidor) y de sesión (afectan a las conexiones). El comando SET permite modificar el valor de una variable global o de sesión.

Sintaxis:

```
SET GLOBAL nombre_variable=valor
SET @@global.nombre_variable=valor
SET [SESSION] nombre_variable=valor
SET [@@[session.]]nombre_variable=valor
```

Se puede consultar más información y todas las variables disponibles en:

<https://mariadb.com/kb/en/server-status-variables/>
<https://mariadb.com/kb/en/server-system-variables/>

- **Modos SQL.** MariaDB admite varios modos diferentes que le permiten ajustarlo para que se adapte a sus necesidades. La variable *SQL_MODE* se usa para hacer que MariaDB emule el comportamiento de otros servidores SQL. *SQL_MODE* es una cadena con diferentes opciones separadas por coma, sin espacios.

El valor por defecto en

Sintaxis:

```
-- Comprobar el valor de modo local (de sesión) y global.
SELECT @@SQL_MODE, @@GLOBAL.SQL_MODE;
-- Establecer nuevos valores para el modo.
SET sql_mode = 'modes';
SET GLOBAL sql_mode = 'modes';
```

Se puede consultar más información y todos los modos disponibles en:

<https://mariadb.com/kb/en/sql-mode/>
https://mariadb.com/docs/reference/es/system-variables/sql_mode/

3. Sintaxis utilizada

Es la sintaxis usada para indicar la forma general de un comando, indicando todas las posibles opciones. A la hora de usar el comando, se utilizarán las opciones convenientes y sobre los objetos pertinentes.

Ejemplo: `SELECT * | {[ALL|DISTINCT] columna | expresión [alias],...} FROM tabla;`

Para indicar la sintaxis de un comando se usan símbolos especiales. Los **símbolos** que utilizaremos (de acuerdo con la sintaxis que se utiliza normalmente en cualquier documentación de este tipo) son:

- **PALABRA** - Cuando en la sintaxis se utiliza una palabra en mayúsculas, significa que es una palabra que hay que escribir literalmente (aunque sin importar si en mayúsculas o minúsculas).
- **Texto** - El texto que aparece en minúsculas sirve para indicar que habrá que sustituirlo por un objeto de ese tipo.
Ejemplo: `SELECT columna FROM tabla;`
El texto *columna* hay que cambiarlo por un nombre concreto de columna (nombre, apellidos...), al igual que *tabla* se refiere a un nombre de tabla concreto.
- **[] (corchetes)** - Sirven para encerrar texto que no es obligatorio en el comando, es decir para indicar una parte opcional.
- **| (barra vertical)** - Indica opción. Las palabras separadas con este signo indican que se debe elegir una de entre todas las palabras.
- **... (puntos suspensivos)** - Indica que se puede repetir lo anterior.
- **{ } (llaves)** - Sirven para indicar opciones mutuamente exclusivas pero obligatorias. Es decir, opciones de las que sólo se puede elegir una opción, pero de las que es obligado elegir una.
Ejemplo: `SELECT { * | columna | expresión } FROM tabla;`
El ejemplo anterior indicaría que se debe elegir obligatoriamente el asterisco o un nombre de columna o una expresión. Si las llaves del ejemplo fueran corchetes, entonces indicarían que incluso podría no aparecer ninguna opción.

Consideraciones importantes que debemos tener en cuenta:

- En SQL no se distingue entre mayúsculas y minúsculas.
 - Las bases de datos y las tablas se corresponden a carpetas y archivos, por lo que los nombres de bases de datos y tablas tienen en cuenta mayúsculas y minúsculas si el sistema operativo también las tiene en cuenta para los nombres de archivo (en Linux, sí distingue; en Windows, irrelevante).
- Las instrucciones finalizan con el signo de punto y coma. Cualquier comando SQL (`SELECT`, `INSERT`...) puede ser partido por espacios o saltos de línea antes de finalizar la instrucción.
- A la hora de nombrar objetos (bases de datos, tablas, columnas...) es recomendable utilizar sólo caracteres sin acentos, cifras y los signos \$ y (subrayado).

- Los comentarios multilínea comienzan por /* y terminan por */. Los comentarios de una única línea empiezan con los símbolos # ó -- (seguido de un espacio en blanco).

Esto es un comentario

-- Esto es un comentario

4. Tipos de datos

A la hora de crear tablas, hay que indicar el tipo de datos de cada campo. Cada SGBD ha creado tipos específicos para necesidades precisas. El tipo elegido dependerá de la precisión buscada, teniendo en cuenta el tamaño necesario para almacenar datos. Una buena decisión permite acceder rápidamente al valor.

Todos los tipos de datos que maneja MariaDB se pueden consultar en su documentación oficial (<https://mariadb.com/kb/en/data-types/>).

Los más destacados son:

TIPO	DESCRIPCIÓN
Texto	
CHAR [(M)]	Cadenas de caracteres de longitud fija M (hasta 255). Por defecto 1. Cuando se almacena, la cadena se completa a la derecha con espacios hasta la longitud definida; estos espacios son automáticamente eliminados cuando la cadena se lee.
VARCHAR (M)	Cadenas de caracteres de longitud variable, máximo M (hasta 65535). Cuando se almacena, no se añade ningún espacio extra.
TINYTEXT	Columna de texto con un máximo de 255 (2^8-1) caracteres. Equivalente a VARCHAR(255).
TEXT [(M)]	Columna de texto con un máximo de 65535 ($2^{16}-1$) caracteres, pudiéndose especificar el tamaño M que se desee.
MEDIUMTEXT	Columna de texto con un máximo de 16.777.215 ($2^{24}-1$) caracteres.
LONGTEXT	Columna de texto con un máximo de 4.294.967.295 ($2^{32}-1$) caracteres.
ENUM ('valor1', 'valor2', ...)	Enumeración. Cadena que puede seleccionar un único valor, elegido de la lista de valores que se adjunta. Internamente, los valores ENUM son representado como enteros.
SET ('valor1', 'valor2', ...)	Conjunto. Cadena que puede seleccionar cero o más valores, cada uno de los cuales son escogidos de la lista de valores que se adjunta. Internamente, los valores SET son representados como enteros.
Números	
TINYINT	Entero muy pequeño. Ocupa 1 byte. Valores, de -128 a 127 (si es UNSIGNED, de 0 a 255).
SMALLINT	Entero pequeño. Ocupa 2 bytes. Valores, de -32768 a 32767 (si es UNSIGNED, de 0 a 65535).
MEDIUMINT	Entero mediano. Ocupa 3 bytes. Valores, de

	-8388608 a 8388607 (si es UNSIGNED, de 0 a 16777215).																					
INT - INTEGER	Entero normal. Ocupa 4 bytes. Valores, de -2.147.483.648 a 2.147.483.647 (si es UNSIGNED, de 0 a 4.294.967.295).																					
BIGINT	Entero largo. Ocupa 8 bytes.																					
DECIMAL [(M [,D])] DEC, NUMERIC y FIXED son sinónimos a DECIMAL A partir de MariaDB 10.3, NUMBER también es sinónimo a DECIMAL	Número en punto fijo (valores exactos). Utilizados cuando es necesario guardar los valores exactos sin redondeos (por ejemplo, para datos monetarios). M indica el número de dígitos en total (la precisión), con un rango de 1 a 65. D indica el número de cifras decimales, con un rango de 0 a 38 (desde Mariadb 10.2.1; antes a 30). Si M se omite, por defecto se toma el valor 10. Si D se omite, por defecto se toma el valor 0. Ejemplo: <table><tr><th>Formato</th><th>Número escrito por el usuario</th><th>Se almacena como...</th></tr><tr><td>DECIMAL</td><td>7,456,123.89</td><td>7456123.89</td></tr><tr><td>DECIMAL (9)</td><td>7,456,123.89</td><td>7456124</td></tr><tr><td>DECIMAL (9,2)</td><td>7,456,123.89</td><td>7456123.89</td></tr><tr><td>DECIMAL (9,1)</td><td>7,456,123.89</td><td>7456123.9</td></tr><tr><td>DECIMAL (6)</td><td>7,456,123.89</td><td>Da error de precisión</td></tr><tr><td>DECIMAL (7,-2)</td><td>7,456,123.89</td><td>7456100</td></tr></table>	Formato	Número escrito por el usuario	Se almacena como...	DECIMAL	7,456,123.89	7456123.89	DECIMAL (9)	7,456,123.89	7456124	DECIMAL (9,2)	7,456,123.89	7456123.89	DECIMAL (9,1)	7,456,123.89	7456123.9	DECIMAL (6)	7,456,123.89	Da error de precisión	DECIMAL (7,-2)	7,456,123.89	7456100
Formato	Número escrito por el usuario	Se almacena como...																				
DECIMAL	7,456,123.89	7456123.89																				
DECIMAL (9)	7,456,123.89	7456124																				
DECIMAL (9,2)	7,456,123.89	7456123.89																				
DECIMAL (9,1)	7,456,123.89	7456123.9																				
DECIMAL (6)	7,456,123.89	Da error de precisión																				
DECIMAL (7,-2)	7,456,123.89	7456100																				
FLOAT [(M,D)]	Número en punto flotante (valores aproximados) de 4 bytes. M indica el número de dígitos en total (precisión) y D es el número de cifras decimales. Si se omiten M y D, los valores son almacenados a los límites permitidos por el hardware.																					
DOUBLE [(M,D)] REAL es sinónimo a DOUBLE	Número en punto flotante de 8 bytes. M indica el número de dígitos en total (precisión) y D es el número de cifras decimales.																					
BIT [(M)]	Campo tipo bit. M indica el número de bits por valor, de 1 a 64. Si M se omite, por defecto es 1.																					
La mayoría de los tipos numéricos pueden utilizar los modificadores SIGNED (con signo, por defecto), UNSIGNED (sin signo) o ZEROFILL (rellena con ceros por la izquierda).																						
Booleanos																						
BOOL – BOOLEAN	Equivalente a TINYINT(1). Un valor de cero es considerado FALSE. Un valor no cero es considerado TRUE.																					
Fechas																						
DATE	Fecha. Formato por defecto, 'YYYY-MM-DD'; otros, 'YY-MM-DD', YYMMDD, 'YYYY/MM/DD'...																					
TIME [(<precisión microsecs>)]	Hora. El rango es de '-838:59:59.999999' a '838:59:59.999999'. El formato en el que se muestra por defecto es 'HH:MM:SS.ssssss'; otros, 'D HH:MM:SS'.																					

	'HH:MM:SS', 'HH:MM', 'D HH:MM', 'D HH', 'SS', or 'HHMMSS'...
DATETIME	Fecha y hora en formato 'YYYY-MM-DD HH:MM:SS.ffffff'. Desde MariaDB 10.3, DATE con una porción de tiempo es sinónimo del tipo DATETIME. Rango posible desde 1000-01-01 00:00:00.000000 a 9999-12-31 23:59:59.999999.
TIMESTAMP	Fecha y hora en formato 'YYYY-MM-DD HH:MM:SS.ffffff'. Este campo se usa generalmente para definir en qué momento se agregó o actualizó una fila y, de manera predeterminada, se le asignará automáticamente la fecha y hora actual cuando se inserte o actualice un registro. Rango posible desde '1970-01-01 00:00:01' a '2038-01-19 03:14:07'.
YEAR [(4)]	Año en formato de 2 ó 4 dígitos (por defecto, 4). El formato de 2 dígitos se considera obsoleto. En el formato de 4 dígitos, el rango permitido es desde 1901 a 2155.
Para almacenar la fecha actual, la mayoría de los SGBD proporcionan funciones, como SYSDATE(), que devuelven ese valor.	
Datos de gran tamaño	
BLOB [(M)]	Objeto binario de un máximo de 65535 ($2^{16}-1$) bytes. Cada valor BLOB se almacena utilizando un prefijo de 2 bytes de longitud que indica el número de bytes en el valor. Si se añade el valor M, MariaDB crea una columna con un tipo BLOB lo suficientemente grande como para almacenar valores de M bytes de longitud.
MEDIUMBLOB	Objeto binario de un máximo de $2^{16}-1$ bytes. Cada valor BLOB se almacena utilizando un prefijo de 3 bytes de longitud que indica el número de bytes en el valor.
LOB	Objeto binario de un máximo de $2^{32}-1$ bytes. Cada valor BLOB se almacena utilizando un prefijo de 4 bytes de longitud que indica el número de bytes en el valor.

5. Consulta de datos: SELECT

El único comando que se usa para realizar consultas es el versátil comando **SELECT**. Da la posibilidad de seleccionar ciertas columnas de una tabla, seleccionar ciertas filas de acuerdo con ciertos criterios, mezclar datos de tablas diferentes, realizar cálculos sobre los datos e incluso agrupar datos y realizar cálculos sobre los grupos. Se pueden realizar consultas complejas, con numerosas tablas, pero siempre hay que poner atención al rendimiento, que puede disminuir rápidamente en un comando SQL mal construido o que no utilice los índices correctos en las tablas.

El formato general para realizar una consulta con la sentencia SELECT en MariaDB sería:

```
SELECT
  [ALL | DISTINCT | DISTINCTROW]
  [HIGH_PRIORITY]
  [STRAIGHT_JOIN]
  [SQL_SMALL_RESULT] [SQL_BIG_RESULT] [SQL_BUFFER_RESULT]
  [SQL_CACHE | SQL_NO_CACHE] [SQL_CALC_FOUND_ROWS]
  select_expr [, select_expr ...]
  [ FROM table_references
    [WHERE where_condition]
    [GROUP BY {col_name | expr | position} [ASC | DESC], ... [WITH ROLLUP]]
    [HAVING where_condition]
    [ORDER BY {col_name | expr | position} [ASC | DESC], ...]
    [LIMIT {[offset,] row_count | row_count OFFSET offset}]
    procedure|[PROCEDURE procedure_name(argument_list)]
    [INTO OUTFILE 'file_name' [CHARACTER SET charset_name] [export_options]
```

Para empezar con consultas sencillas, se puede simplificar la completa definición anterior y quedarnos con:

```
SELECT
  [DISTINCT] select_expr [, select_expr ...]
  [ FROM table_references
    [WHERE where_condition]
    [GROUP BY {col_name | expr | position} [ASC | DESC], ... [WITH ROLLUP]]
    [HAVING where_condition]
    [ORDER BY {col_name | expr | position} [ASC | DESC], ...]
    [LIMIT {[offset,] row_count | row_count OFFSET offset}]
```

Es muy importante conocer en qué orden se ejecuta cada una de las cláusulas que forman la sentencia SELECT. El orden de ejecución, su opcionalidad y su explicación se detalla en la siguiente tabla:

Cláusula	Obligatoria / Opcional	Expresión
FROM	Obligatoria	Tabla(s) fuente(s)
WHERE	Opcional	Condición (condiciones) o restricción (restricciones)
GROUP BY	Opcional	Agrupación (agrupaciones)
HAVING	Opcional	Condición (condiciones) o restricción (restricciones) sobre la(s) agrupación (agrupaciones)
SELECT	Obligatoria	Lista columna(s) y/o elementos de extracción
ORDER BY	Opcional	Ordenación (ordenaciones)
LIMIT	Opcional	Limitar el número de filas devueltas en la consulta

El resultado de una consulta siempre será una tabla de datos, que puede tener una o varias columnas y ninguna, una o varias filas.

A continuación, se describirán en más detalle cada una de estas cláusulas, excepto GROUP BY y HAVING, que se trabajarán en temas posteriores.

5.1. CLÁUSULA SELECT

Permite indicar cuáles serán las columnas que tendrá la tabla de resultado de la consulta que se está realizando. Las opciones que se pueden indicar son:

- El nombre de una o varias columnas (separadas por coma) de la tabla sobre la que estamos realizando la consulta.
- Una constante que aparecerá en todas las filas de la tabla resultado.
- Una expresión que nos permite calcular nuevos valores.

Las particularidades más destacadas de esta cláusula son los siguientes:

- *** (asterisco).** Significa que se seleccionan todas las columnas de las tablas elegidas. Frente a todas las columnas de la tabla origen, también podemos seleccionar sólo algunas columnas, separando dichas columnas por comas.

Ejemplos:

```
/* Selección de todos los registros de la tabla alumno */  
SELECT * FROM alumno;  
/* Selección de algunos campos*/  
SELECT apellido1, nombre FROM alumno;
```

- **Select expr.** Es el nombre de una columna (o una expresión formada a partir de las columnas) de las tablas que se desea mostrar en la salida del SELECT.

Ejemplos:

```
SELECT apellido1 FROM alumno;  
SELECT concat(nombre, ' ', apellido1) FROM alumno; -- concatena cadenas
```

- **ALL.** Se muestran todas las filas de la salida, aunque estén duplicadas (es la opción por defecto).
- **DISTINCT.** Hace que no se muestren las filas duplicadas en la salida.

Compara la salida de (resultados de izquierda (1) a derecha (3)):

- 1) `SELECT apellido1 FROM alumno;`
- 2) `SELECT ALL apellido1 FROM alumno;`
- 3) `SELECT DISTINCT apellido1 FROM alumno;`

apellido1	apellido1	apellido1
▶ Sánchez	▶ Sánchez	▶ Sánchez
Sáez	Sáez	Sáez
Ramírez	Ramírez	Ramírez
Sánchez	Sánchez	Martínez
Martínez	Martínez	Gutiérrez
Gutiérrez	Gutiérrez	Fernández
Fernández	Fernández	Carretero
Carretero	Carretero	Domínguez
Domínguez	Domínguez	Moreno
Moreno	Moreno	

La cláusula **DISTINCT** no se puede utilizar con operaciones de agrupamiento (GROUP BY, que se verá en temas posteriores). De hecho, los operadores de tipo COUNT o SUM eliminan automáticamente los duplicados.

- **alias.** En una consulta SQL que contenga varias tablas, es preferible asignar un diminutivo (alias) a cada nombre de tabla, siempre bajo la óptica de hacer que las consultas sean más legibles para todos los programadores. Este alias se puede utilizar para un nombre de una columna y también para un resultado de una función o un SELECT anidado. Debe escribirse tal cual o entre comillas simples si contiene blancos. La palabra clave AS puede omitirse.

Expre.columna [AS] alias

Ejemplos:

SELECT A.nombre, A.apellido1 FROM alumno A;

SELECT concat nombre,' ', apellido1 AS 'nombre compuesto' FROM alumno;

nombre compuesto
María Sánchez
Juan Sáez
Pepe Ramírez

Nota: Los alias no se pueden usar en los WHERE, pero sí en el ORDER BY

- **Expresiones numéricas.** Los operadores **+** (suma), **-** (resta), ***** (multiplicación), **/** (división) y **%** (módulo – devuelve el resto entero de una división), se pueden utilizar para hacer cálculos numéricos en las consultas.

Ejemplos:

*SELECT apellido, salario, salario*1.16 FROM Emple;*

Esa consulta obtiene tres columnas. La tercera tendrá como nombre la expresión utilizada, para poner un alias basta utilizar dicho alias tras la expresión:

*SELECT apellido, salario, salario*1.16 AS salario_con_bonus FROM Emple;*

apellido	salario	salario_con_bonus
SANCHEZ	1040	1206.40
ARROYO	1500	1740.00
SALA	1625	1885.00
JIMENEZ	2900	3364.00
MARTIN	1600	1856.00
NEGRO	3005	3485.80
CEREZO	2885	3346.60

La prioridad de esos operadores es la normal:

- más prioridad la multiplicación y división * /
- menos prioridad la suma y la resta + -

En caso de igualdad de prioridad, se realiza primero la operación que esté más a la izquierda. Se puede alterar esa prioridad usando paréntesis, ejecutándose primero lo que esté en el interior de los paréntesis.

Cuando una expresión aritmética se calcula sobre valores *NULL*, el resultado es el propio valor *NULL*.

Ejemplos:

SELECT salario, comision FROM Emple;

SELECT salario+comision FROM Emple;

salario	comision
1040	NULL
1500	390
1625	650
2900	NULL
1600	1020
3005	NULL

salario+comision
NULL
1890
2275
NULL
2620
NULL

- **Funciones.** Es posible hacer uso de funciones específicas de SQL en la cláusula *SELECT*. Por ejemplo, como se ha visto en la página anterior, la función *CONCAT* se utiliza para concatenar dos cadenas.

Se verán en detalle en un tema posterior.

5.2. CLÁUSULA FROM

Permite indicar la tabla o tablas de las que se sacarán los datos. Si hay más de una tabla se realiza el producto cartesiano de las tablas, es decir, se combinan todas las filas de una tabla con todas las de la(s) otra(s) tabla(s). Si se desea, detrás del nombre de tabla se puede poner un alias y usarlo dentro de la consulta.

5.3. CLÁUSULA WHERE

Sirve para restringir los datos de salida de las tablas dando una condición que han de cumplir todos los registros de la salida.

Se pueden diferenciar varios tipos de condiciones que pueden aparecer en la cláusula *WHERE*:

- Condiciones para comparar valores o expresiones.
- Condiciones para comprobar si un valor está dentro de un rango de valores.
- Condiciones para comprobar si un valor está dentro de un conjunto de valores.
- Condiciones para comparar cadenas con patrones.
- Condiciones para comprobar si una columna tiene valores a *NULL*.

Ejemplo:

SELECT nombre FROM alumno WHERE es_repetidor='no';

Los operandos usados en las condiciones pueden ser nombres de columnas, constantes o expresiones. Los operadores que podemos usar en las condiciones pueden ser aritméticos, de comparación, lógicos, etc.

La lista completa de operadores que se pueden utilizar en MariaDB se puede consultar en <https://mariadb.com/kb/en/operators/>. En este documento se estudiarán los más utilizados.

Operadores aritméticos

Operador	Significado
+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo

Operadores de comparación

Operador	Significado
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que
=	Igual que
<>	Distinto
!=	Distinto

Se pueden utilizar tanto para comparar números como para comparar textos y fechas. En el caso de los textos, las comparaciones se hacen en orden alfabético (orden alfabético estricto, es el orden del código ASCII).

Ejemplos:

```
SELECT nombre, apellido1, apellido2 FROM alumnos WHERE
apellido1<apellido2;
```

nombre	apellido1	apellido2
Juan	Sáez	Vega
Irene	Gutiérrez	Sánchez
Cristina	Fernández	Ramírez
Antonio	Carretero	Ortega
Manuel	Domínguez	Hernández
Daniel	Moreno	Ruiz

```
SELECT nombre, apellido1, apellido2, fecha_nacimiento FROM alumno WHERE
fecha_nacimiento<'1995-12-31';
```

nombre	apellido1	apellido2	fecha_nacimiento
María	Sánchez	Pérez	1990-12-01
Pepe	Ramírez	Gea	1988-01-03
Lucía	Sánchez	Ortega	1993-06-13
Paco	Martínez	López	1995-11-24
Irene	Gutiérrez	Sánchez	1991-03-28
Antonio	Carretero	Ortega	1994-05-20

Operadores lógicos

Operador	Significado	Descripción
AND	Y lógica	Devuelve verdadero si las expresiones a su izquierda y derecha son ambas verdaderas
&&	Y lógica	
OR	O lógica	Devuelve verdadero si cualquiera de las dos expresiones a izquierda y derecha del OR, son verdaderas
	O lógica	
NOT	Negación lógica	Invierte la lógica de la expresión que está a su derecha. Si era verdadera, mediante NOT pasa a ser falso y viceversa
!	Negación lógica	

Ejemplos:

/ Obtiene a las personas de entre 25 y 50 años*/*
SELECT nombre, apellido1, apellido2 FROM personas
WHERE edad>=25 AND edad<=50;

*/*Obtiene a la gente de más de 60 años o de menos de 20*/*
SELECT nombre, apellido1, apellido2 FROM personas
WHERE edad>60 OR edad<20;

*/*Obtiene a las personas con primer apellido entre la A y la O */*
SELECT nombre, apellido1, apellido2 FROM personas
WHERE apellido1>'A' AND apellido1<'P';

*** EJERCICIOS - Realizar los ejercicios 1-12 ***

Operador BETWEEN

Nos permite obtener datos que se encuentren en un rango. Los valores inicial y final están incluidos.

Sintaxis: *Expresion [NOT] BETWEEN valor_inicial AND valor_final;*

Ejemplos:

*/*Obtiene los alumnos que tienen entre 1 y 4 faltas en el trimestre 1 (faltas1)*/*
SELECT nombre, apellidos FROM Alum1213 WHERE faltas1 BETWEEN 1 AND 4;
*/*Obtiene los pedidos que se han realizado durante el mes de enero de 2018*/*
*SELECT * FROM pedido WHERE fecha_pedido BETWEEN '2018-01-01' AND '2018-01-31';*

Operador IN

Para un conjunto de valores posibles, permite obtener registros cuyos valores estén en una lista de valores. Lógicamente, también puede comprobar si el valor de una determinada columna está incluido en una lista de valores.

Sintaxis: *Expresion [NOT] IN (lista de valores separados por ,);*

Ejemplos:

```
/*Obtiene piezas cuyos precios sean 3, 5 u 8 (no valen ni el precio 4 ni el 6)*/  
SELECT tipo, modelo, precio FROM piezas WHERE precio IN (3,5, 8);  
/*Obtiene todos los datos de los alumnos que no tengan como primer apellido  
Sánchez, Martínez o Domínguez*/  
SELECT * FROM alumno  
WHERE apellido1 NOT IN ('Sánchez', 'Martínez', 'Domínguez');
```

Operador LIKE

Se utiliza para comparar si una cadena de caracteres coincide con un patrón. En el patrón podemos utilizar cualquier carácter alfanumérico, pero hay dos caracteres que tienen un significado especial, el símbolo del porcentaje (%) y el carácter de subrayado (_).

- % → Equivale a una serie cualquiera de caracteres (puede ser 0 caracteres).
- _ → Equivale a un carácter cualquiera (siempre un carácter en su lugar).

En los patrones de comparación, hay distinción entre mayúsculas y minúsculas.

Sintaxis: *Expresion [NOT] LIKE patrón;*

Ejemplos:

```
/* Selecciona nombres que empiecen por S */  
SELECT nombre FROM personas WHERE nombre LIKE 'S%';  
/* Selecciona las personas cuyo apellido sea Sanchez, Senchez, Stnchez,... */  
SELECT apellido1 FROM Personas WHERE apellido1 LIKE 'S_nchez';
```

Operador REGEXP y expresiones regulares

El operador REGEXP permite realizar búsquedas mucho más potentes que LIKE haciendo uso de expresiones regulares.

Sintaxis: *expr REGEXP pat*

Devuelve 1 si *expr* coincide con *pat* o 0 si no coincide.

No es necesario que el patrón sea una cadena literal; también se puede especificar una expresión de cadena o una columna de una tabla.

Ejemplos:

```
SELECT 'Maria' REGEXP 'Mari';  
→ Devolvería 1, pues el patrón 'Mari' existe en la expresión 'Maria'  
SELECT 'Mari' REGEXP 'Maria';
```

→ Devolvería 0. En este caso, el patron 'Maria' no existe en la expresión 'Mari'

Al ser un operador un tanto avanzado pudiendo ser utilizando junto a una serie de caracteres especiales (^ (comparación al comienzo de cadena), \$ (comparación al final de cadena), . (comparar un único carácter), * (comparar cero o más caracteres), etc), se puede consultar más información en:

- <https://mariadb.com/kb/en/regular-expressions-overview/>
- <https://mariadb.com/kb/en/regexp/>

Operadores IS NULL

Devuelve verdadero si el valor que examina es nulo:

Sintaxis: *Expresion IS [NOT] NULL*

Ejemplo:

*/*Seleccionar a la gente que no tiene teléfono*/*

SELECT nombre, apellidos FROM personas WHERE telefono IS NULL

Prioridad en los operadores

A veces, las expresiones que se producen en los **SELECT** son muy extensas y es difícil saber qué parte de la expresión se evalúa primero. Por ello se indica la siguiente tabla de precedencia (<https://mariadb.com/kb/en/operator-precedence>), con los operadores básicos:

Orden de precedencia	Operador
1	*(multiplicar) / (dividir) % (módulo)
2	- (resta) + (suma)
3	Comparaciones (>, <, !=, ...)
4	IS NULL, LIKE, REGEXP, IN
5	BETWEEN
6	NOT
7	AND
8	OR
9	= (asignación)

5.4. CLÁUSULA ORDER BY

El orden inicial de los registros obtenidos por un SELECT no guarda más que una relación respecto al orden en el que fueron introducidos. Para ordenar en base a otros criterios se utiliza la cláusula ORDER BY.

Se puede ordenar sobre cualquier columna de una tabla; para ello, es necesario que las columnas formen parte de la selección, pero no es obligatorio que se muestren.

Sintaxis: *ORDER BY {col_name | expr | position} [ASC | DESC], ...*

En esa cláusula se coloca una lista de campos que indica la forma de ordenar. Se ordena primero por el primer campo de la lista; si hay coincidencias, por el segundo; si ahí también las hay, por el tercero, y así sucesivamente.

Se pueden colocar las palabras ASC O DESC en cada columna de ordenación para indicar si se desea orden ascendente o descendente (por defecto se toma ASC – del más pequeño al más grande; en fechas, de fecha más lejana a más cercana).

La ordenación puede ser origen de un bajo rendimiento de la consulta. De hecho, si la consulta recupera millones de filas, el sistema va a recuperar todas las filas en un primer momento, para posteriormente ordenar el conjunto y solo después empezar a restituir los elementos en el orden oportuno. Por tanto, se desaconseja el uso de esta operación en caso de que no haya cláusula WHERE o cuando ésta es poco restrictiva y la tabla contiene millones de registros.

Ejemplos:

/ Muestra los alumnos ordenados por su teléfono */*

-- Imagen izquierda

SELECT nombre, apellido1, teléfono FROM alumno ORDER BY teléfono;

SELECT nombre, apellido1, teléfono FROM alumno ORDER BY 3;

-- Imagen derecha

SELECT nombre, apellido1 FROM alumno ORDER BY teléfono;

nombre	apellido1	teléfono
María	Sánchez	NULL
Manuel	Domínguez	NULL
Irene	Gutiérrez	NULL
Pepe	Ramírez	NULL
Daniel	Moreno	NULL
Antonio	Carretero	612345633
Juan	Sáez	618253876
Cristina	Fernández	628349590
Lucía	Sánchez	678516294
Paco	Martínez	692735409

nombre	apellido1
María	Sánchez
Manuel	Domínguez
Irene	Gutiérrez
Pepe	Ramírez
Daniel	Moreno
Antonio	Carretero
Juan	Sáez
Cristina	Fernández
Lucía	Sánchez
Paco	Martínez

/ Muestra los alumnos ordenados en orden ascendente de apellido y descendente de nombre */*

SELECT nombre, apellido1 FROM alumno ORDER BY apellido1, nombre DESC;

/ Muestra los alumnos por fecha de nacimiento (de mayores a más jóvenes) */*

SELECT nombre, fecha_nacimiento FROM alumno ORDER BY 2;

nombre	fecha_nacimiento
Pepe	1988-01-03
María	1990-12-01
Irene	1991-03-28
Lucía	1993-06-13
Antonio	1994-05-20
Paco	1995-11-24
Cristina	1996-09-17
Daniel	1998-02-03
Juan	1998-04-02
Manuel	1999-07-08

5.5. CLÁUSULA LIMIT

La cláusula LIMIT permite limitar el número de filas que se incluyen en el resultado de la consulta.

Sintaxis: `[LIMIT {[offset,] row_COUNT | row_COUNT OFFSET offset}]`

row_COUNT es el número de filas que queremos obtener y *offset* el número de filas que nos saltamos antes de empezar a contar. Es decir, la primera fila que se obtiene como resultado es la que está situada en la posición *offset+1*.

Ejemplo:

/ A partir del 5º alumno más joven, muestra los 3 siguientes por edad */*

SELECT nombre, fecha_nacimiento FROM alumno ORDER BY 2 LIMIT 5, 3;

-- Sería equivalente a la consulta:

SELECT nombre, fecha_nacimiento FROM alumno ORDER BY 2 LIMIT 3 OFFSET 5;

-- Ver la tabla resultado del ejemplo anterior para comprobarlo

nombre	fecha_nacimiento
Paco	1995-11-24
Cristina	1996-09-17
Daniel	1998-02-03

*** EJERCICIOS - Realizar los ejercicios 13-25 ***

6. Combinación de tablas (consultas multitabla)

Es más que habitual necesitar en una consulta datos que se encuentran distribuidos en varias tablas. Las bases de datos relacionales se basan en que los datos se distribuyen en tablas que se pueden relacionar mediante un campo. Ese campo es el que permite integrar los datos de las diferentes tablas.

Por ejemplo, si disponemos de una tabla de empleados cuya clave es el *dni* y otra tabla de tareas que se refiere a tareas realizadas por los empleados, es seguro (si el diseño está bien hecho) que en la tabla de tareas aparecerá el *dni* del empleado para saber qué empleado realizó la tarea.

En el ejemplo anterior, si quiere obtener una lista de los datos de las tareas y los empleados, se podría hacer de esta forma:

```
SELECT cod_tarea, descripcion_tarea, dni_empleado, nombre_empleado
FROM tareas, empleados;
```

La sintaxis es correcta ya que, efectivamente, en el apartado *FROM* se pueden indicar varias tablas separadas por comas. Sin embargo, esto produce un producto cruzado o producto cartesiano, es decir, aparecerán todos los registros de las tareas combinados con todos los registros de empleados, es decir, devolverá para cada fila de la primera tabla todas las filas del resto de tablas, y así sucesivamente (si tenemos 3 y 2 filas en cada tabla, el sistema devolverá $3 \times 2 = 6$ filas) tal y como se puede apreciar en la siguiente imagen.

Tabla 1	
campo 1	campo 2
a1	b1
a2	b2
a3	b3

Tabla 2	
campo 11	campo 22
a4	b4
a5	b5

Tabla1 X Tabla 2			
campo 1	campo 2	campo 11	campo 22
a1	b1	a4	b4
a1	b1	a5	b5
a2	b2	a4	b4
a2	b2	a5	b5
a3	b3	a4	b4
a3	b3	a5	b5

*** EJERCICIO - Realizar el ejercicio 26 ***

El producto cartesiano a veces es útil para realizar consultas complejas, pero en el caso normal no lo es ya que lo lógico sería “vincular” las tablas entre ellas para obtener un resultado conforme a lo que esperamos y que sea realista, es decir, para este caso particular, necesitamos filtrar ese producto para que sólo aparezcan los registros de las tareas relacionadas con sus empleados correspondientes. A eso se le llama combinar o asociar (**JOIN**) tablas.

Para realizar este tipo de consultas podemos usar dos alternativas:

1. La sintaxis de SQL1 (SQL-86), que consiste en realizar el producto cartesiano de las tablas y añadir un filtro para relacionar los datos que tienen en común.
2. La sintaxis de SQL2 (SQL-92 y SQL-2003) que incluye todas las cláusulas de tipo JOIN.

Alternativa 1 – Sintaxis SQL1

La forma de realizar correctamente la consulta anterior asociando las tareas con los empleados que la realizaron sería indicando la relación existente entre las diferentes tablas (a través de la clave foránea) en la cláusula WHERE:

```
SELECT cod_tarea, descripc, dni_emple, nombre
FROM tareas, empleados
WHERE tareas.dni_emple = empleados.dni;
```

EMPLEADOS			TAREAS		
dni	nombre	tfno	cod_tarea	descripc	dni_emple
12345689Y	María	941222832	01	Supervisar	12345689Y
98765432F	Juan	605666845	02	Diseñar	98765432F
72732827K	Sheila	941292928	03	Analizar	72732827K
****	****	****	04	Programar	72732827K
****	****	****	05	Arreglar	98765432F

Nótese que se utiliza la notación **tabla.columna** para evitar ambigüedades, ya que el mismo nombre de campo se puede repetir en ambas tablas (si en vez de dni_emple se especificara sólo dni). Si algunas columnas tienen el mismo nombre perteneciendo a tablas distintas, habrá que añadir el nombre de la tabla delante de la columna, para que el sistema sepa qué columna debe elegir.

No es obligatorio poner el nombre de las tablas delante de cada columna, pero por una cuestión de legibilidad y de mantenimiento, es preferible ponerlas cuando se trate de selecciones complejas.

Para evitar repetir continuamente el nombre de la tabla, se puede utilizar un alias de tabla:

```
SELECT a.cod_tarea, a.descripcion_tarea, b.dni_empleado,  
       b.nombre_empleado  
FROM tareas a, empleados b  
WHERE a.dni_empleado = b.dni;
```

Al apartado WHERE se le pueden añadir condiciones, encadenándolas con el operador AND.

Ejemplo:

```
SELECT a.cod_tarea, a.descripcion_tarea  
FROM tareas a, empleados b  
WHERE a.dni_empleado = b.dni AND b.nombre_empleado='Javier';
```

Finalmente, también se pueden enlazar más de dos tablas a través de sus campos relacionados.

Ejemplo:

```
SELECT a.cod_tarea, a.descripcion_tarea, b.nombre_empleado,  
       c.nombre_utensilio  
FROM tareas a, empleados b, utensilios_utilizados c  
WHERE a.dni_empleado = b.dni AND a.cod_tarea=c.cod_tarea;
```

A las relaciones descritas anteriormente se las llama relaciones en igualdad (**equijoins**), ya que las tablas se relacionan a través de campos que contienen valores iguales en dos tablas. Son las más frecuentes y se suelen llamar **combinaciones internas**, porque sólo aparecen las filas relacionadas.

*** EJERCICIO - Realizar el ejercicio 27 ***

Alternativa 2 – Sintaxis SQL2

En la versión SQL de 1992 se ideó una nueva sintaxis para consultar varias tablas de forma combinada. La razón fue separar las condiciones de asociación respecto de las condiciones de selección de registros.

La sintaxis completa es:

```
SELECT tabla1.columna1, tabla1.columna2,... tabla2.columna1, tabla2.columna2,...  
FROM tabla1  
    [CROSS JOIN tabla2] |  
    [NATURAL JOIN tabla2] |  
    [JOIN tabla2 USING (columna)] |  
    [[INNER] JOIN tabla2  
        ON (tabla1.columna=tabla2.columna)] |  
    [{LEFT|RIGHT} [OUTER] JOIN tabla2  
        ON (tabla1.columna=tabla2.columna)]  
[WHERE ....] [ORDER BY ...];
```

Se describen sus posibilidades en los siguientes apartados sobre las tablas ejemplo siguientes:

- Alumnos (DNI, Nom_alumno, Ape_alumno, Direccion, Tfno, Fecha_nac, Id_curso↑)
- Cursos (Id_curso, Nom_curso, Tutor, Aula)

- **CROSS JOIN – Producto cartesiano**

Utilizando la opción *CROSS JOIN* se realiza un producto cruzado o cartesiano entre las tablas indicadas. Eso significa que cada tupla de la primera tabla se combina con cada tupla de la segunda tabla. Es decir, si la primera tabla tiene 10 filas y la segunda otras 10, como resultado se obtienen 100 filas, resultado de combinar todas entre sí.

Ejemplo:

```
SELECT * FROM alumnos CROSS JOIN cursos;
```

No es una operación muy utilizada, aunque posibilita resolver consultas extremadamente complicadas. Es igual que poner los nombres de las dos tablas, sin especificar la asociación.

*** EJERCICIO - Realizar el ejercicio 28. Comprueba el resultado con el 26 ***

- **NATURAL JOIN**

Establece una relación de igualdad entre las tablas a través de los campos que tengan el mismo nombre en ambas tablas.

Ejemplo:

```
SELECT * FROM alumnos NATURAL JOIN cursos;
```

En el ejemplo anterior, se obtienen los registros de alumnos relacionados con los cursos a través de los campos que tengan el mismo nombre en ambas tablas (en este caso, id_curso).

Hay que asegurarse de que sólo son las claves principales y secundarias de las tablas relacionadas las columnas en las que el nombre coincide, de otro modo fallaría la asociación y la consulta no funcionaría.

No se aconseja utilizar este tipo de unión, sobre todo por razones de legibilidad. También hay riesgo de que dos columnas tengan el mismo nombre sin pertenecer a una clave, y así el resultado no será el que queremos.

*** EJERCICIO - Realizar el ejercicio 29. Comprueba el resultado con el 27 ***

- **JOIN USING**

Permite establecer relaciones indicando qué columna (o columnas) común a las dos tablas hay que utilizar.

Ejemplo:

```
SELECT * FROM alumnos JOIN cursos USING (id_curso);
```

Las columnas deben de tener exactamente el mismo nombre en ambas tablas, pero si hay otras con el mismo nombre, no las combinará (si, por ejemplo, tanto en alumnos como en cursos se tiene el campo nombre, habría que usar JOIN USING en lugar de NATURAL JOIN).

*** EJERCICIO - Realizar el ejercicio 30. Comprueba el resultado con el 29 ***

- **INNER JOIN – Combinación interna**

Permite establecer relaciones cuya condición se establece manualmente, lo que permite realizar asociaciones más complejas, o bien asociaciones cuyos campos en las tablas no tienen el mismo nombre.

Ejemplo:

```
SELECT * FROM alumnos [INNER] JOIN cursos  
ON (cursos.id_curso=alumnos.id_curso);
```

Utilizando las formas anteriores de relacionar tablas (incluidas las explicadas con la sintaxis SQL 92), sólo aparecen en el resultado de la consulta filas presentes en las tablas relacionadas, es decir, aplicado al ejemplo que tenemos, sólo aparecen alumnos asignados a algún curso. Si hay cursos que no tienen alumnos, esos cursos no aparecen.

*** EJERCICIO - Realizar el ejercicio 31. Comprueba el resultado con el 27 ***

Lo explicado en este apartado para unir dos tablas es igualmente aplicable si se necesitan unir tres o más tablas, como se puede observar en el siguiente ejemplo:

```
SELECT *  
FROM cliente INNER JOIN empleado  
ON cliente.codigo_empleado_rep_ventas = empleado.codigo_empleado  
INNER JOIN pago  
ON cliente.codigo_cliente = pago.codigo_cliente;
```

También es posible realizar una operación de INNER JOIN sobre la misma tabla, para lo que sería necesario utilizar un alias para la tabla, como se puede ver en el siguiente ejemplo:

```
SELECT empleado.nombre, empleado.apellido1, empleado.apellido2, jefe.nombre,  
jefe.apellido1, jefe.apellido2  
FROM empleado INNER JOIN empleado AS jefe  
ON empleado.codigo_jefe = jefe.codigo_empleado;
```

*** EJERCICIOS - Realizar los ejercicios 32 y 33 ***

- **OUTER JOIN - Combinaciones externas**

Si en una combinación de tablas se desea incluir alguna fila de una tabla que no esté relacionada con filas de la otra tabla se utilizan relaciones laterales o externas (**outer join**), existiendo las diferentes posibilidades:

- **LEFT OUTER JOIN.** Devolverá todas las filas de la tabla que hemos colocado a la izquierda de la composición y relacionará las filas de la tabla de la izquierda con las filas de la tabla de la derecha con las que encuentra una coincidencia. Si no encuentra ninguna coincidencia, se mostrarán los valores de la fila de la tabla izquierda y en los valores de la tabla derecha donde no ha encontrado ninguna coincidencia mostrará el valor NULL.
- **RIGHT OUTER JOIN.** Esta consulta devolverá todas las filas de la tabla que hemos colocado a la derecha de la composición y relacionará las filas de la tabla de la derecha con las filas de la tabla de la izquierda con las que encuentre una coincidencia. Si no encuentra ninguna coincidencia, se mostrarán los valores de la fila de la tabla derecha y en los valores de la tabla izquierda donde no ha encontrado una coincidencia mostrará el valor NULL.
- **FULL OUTER JOIN.** Esta opción no está implementada como tal en MariaDB. Sin embargo, se podría simular utilizando el operador UNION (se estudiará en un tema posterior), que nos realiza la unión del resultado de dos consultas. El resultado esperado de una composición FULL OUTER JOIN es obtener la intersección de las dos tablas, junto con las filas de ambas tablas que no se pueden combinar. Dicho con otras palabras, el resultado sería el equivalente a realizar la unión de una consulta de tipo LEFT JOIN y una consulta de tipo RIGHT JOIN sobre las mismas tablas.
- **NATURAL [LEFT | RIGHT] OUTER JOIN.** Esta opción realiza el LEFT ó RIGHT JOIN entre las dos tablas tal y como se ha especificado en los casos anteriores respectivamente, pero, la única diferencia es que en este caso no es necesario utilizar la cláusula ON para indicar sobre qué columna vamos a relacionar las dos tablas. En este caso las tablas se van a relacionar sobre aquellas columnas que tengan el mismo nombre, por lo que hay que tener mucho cuidado al aplicar esta cláusula y asegurarnos bien de que las columnas tienen el mismo nombre.

La sintaxis de los outer join es:

SELECT *columnas*
FROM *tabla1* {**LEFT** | **RIGHT**} [**OUTER**] **JOIN** *tabla2*
ON (*condición*);

Considerando las diferentes opciones sobre un ejemplo práctico, supongamos que tenemos dos tablas: *empleado* y *departamento*, relacionadas por el *id del departamento*, con los siguientes datos.

Tabla: empleado			Tabla: departamento	
id	nombre	id_departamento	id	nombre
1	Pepe	1	1	Desarrollo
2	María	2	2	Sistemas
3	Juan	NULL	3	Recursos Humanos

Si hiciéramos:

```
SELECT * FROM empleado LEFT JOIN departamento
ON empleado.codigo_departamento = departamento.codigo;
```

El resultado de la operación **LEFT JOIN** es:

empleado. id	empleado. nombre	empleado. id_departamento	departamento. id	departamento. nombre
1	Pepe	1	1	Desarrollo
2	María	2	2	Sistemas
3	Juan	NULL	NULL	NULL

Si hiciéramos:

```
SELECT * FROM empleado RIGHT JOIN departamento
ON empleado.codigo_departamento = departamento.codigo;
```

El resultado de la operación **RIGHT JOIN** es:

empleado. id	empleado. nombre	empleado. id_departamento	departamento. id	departamento. nombre
1	Pepe	1	1	Desarrollo
2	María	2	2	Sistemas
NULL	NULL	NULL	3	Recursos Humanos

Si hiciéramos:

```
SELECT * FROM empleado LEFT JOIN departamento
ON empleado.codigo_departamento = departamento.codigo
UNION
SELECT * FROM empleado RIGHT JOIN departamento
ON empleado.codigo_departamento = departamento.codigo;
```

El resultado de la simulación de la operación **FULL JOIN** sería:

empleado. id	empleado. nombre	empleado. id_departamento	departamento. id	departamento. nombre
1	Pepe	1	1	Desarrollo
2	María	2	2	Sistemas
3	Juan	NULL	NULL	NULL
NULL	NULL	NULL	3	Recursos Humanos

Si consideramos otro ejemplo, teniendo las tablas *cursos* y *alumnos*, esta consulta:

```
SELECT * FROM cursos
LEFT OUTER JOIN alumnos
ON (cursos.id_curso=alumnos.id_curso);
```

Obtiene todas las filas de la tabla *cursos* que estén o no relacionadas con datos de la tabla de *alumnos* (la tabla *LEFT* sería *cursos* porque es la que está a la izquierda del *JOIN*). En la salida, los cursos que no tengan alumnos tendrán valores nulos en las columnas de la tabla *alumnos*.

En la consulta anterior, si el *LEFT* lo cambiamos por un *RIGHT*, aparecerán tanto los alumnos que tienen curso asignado como los que no tienen asignado ningún curso.

La condición *FULL OUTER JOIN*, que tal y como se ha indicado no está implementada en MariaDB (pero sí en otros SGBD) produciría un resultado en el que aparecerían los registros no relacionados de ambas tablas (alumnos sin curso y cursos sin alumnos), además de los relacionados, o sea, todos los cursos y alumnos.

Un caso típico de uso de las relaciones externas es el uso de las denominadas **consultas de no coincidentes**. Estas consultas sustituyen de forma más eficiente al operador *NOT IN* ya que permiten comprobar filas de una tabla que no están presentes en una segunda tabla que se relaciona con la primera a través de alguna clave secundaria.

Por ejemplo, supongamos que tenemos una base de datos que posee una tabla de empresas y otra de trabajadores de las empresas. Ambas tablas supongamos que se relacionan a través del CIF de la empresa. Si existen empresas de las que no tenemos ningún trabajador podremos saberlo mediante esta consulta:

```
SELECT e.nombre FROM empresas e
LEFT OUTER JOIN trabajadores t ON (e.cif=t.cif)
WHERE t.dni IS NULL;
```

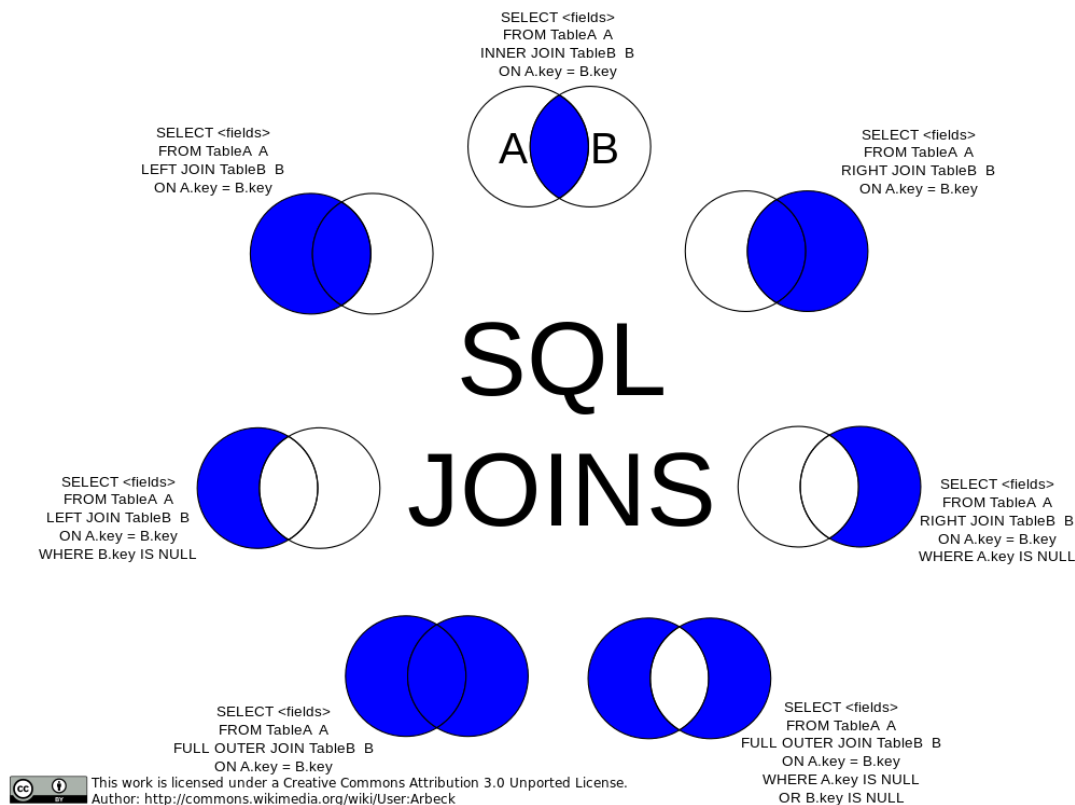
En la consulta anterior gracias a utilizar el operador **LEFT OUTER**, obtenemos la lista completa de empresas (tengan o no trabajadores). Si eliminamos aquellas filas con el *dni* de los trabajadores nulo, entonces salen las empresas sin trabajadores relacionados (sólo las empresas sin trabajadores mostrarán valores nulos en los datos de los trabajadores).

Sobre el ejemplo previo podemos comentar un error común que se puede dar y que es que, cuando se usen *LEFT | RIGHT JOIN* no se deben tener varias condiciones en la cláusula *ON*, es decir, si hubiéramos construido la sentencia anterior como:

```
SELECT e.nombre FROM empresas e
LEFT OUTER JOIN trabajadores t ON (e.cif=t.cif) AND t.dni IS NULL;
```

La consulta sería incorrecta, teniendo que sacar la 2ª condición (*t.dni IS NULL*) a una cláusula *WHERE*, como se hizo inicialmente.

A modo de resumen, la siguiente imagen representa los resultados de aplicar los diferentes tipos de *JOIN*. Hay que tener en cuenta que el *FULL OUTER* no se puede implementar directamente en MariaDB y habría que simularlo con otro tipo de sentencias, tal y como se ha indicado con anterioridad.



*** EJERCICIOS - Realizar los ejercicios 34-37 ***

Se pueden consultar más ejemplos de combinación de tablas en el Anexo A.

Para acabar este apartado, veamos el siguiente ejemplo doble:

```
SELECT *
FROM clients, orders, phoneNumbers
WHERE clients.id = orders.clientId
AND clients.id = phoneNumbers.clientId
AND orderPlaced >= NOW() - INTERVAL 2 WEEK;
```

Frente a:

```
SELECT *
FROM clients INNER JOIN orders ON clients.id = orders.clientId
INNER JOIN phoneNumbers ON clients.id = phoneNumbers.clientId
WHERE orderPlaced >= NOW() - INTERVAL 2 WEEK;
```

Siguiendo la documentación oficial de MariaDB, en su apartado referido a las diferentes formas de escribir estas consultas (según SQL1 (con comas) o SQL2 (con JOIN)), <https://mariadb.com/kb/en/comma-vs-join>, se recomienda utilizar la segunda manera por los siguientes motivos:

- **Legibilidad**, ya que una vez que la cláusula WHERE contiene más de dos condiciones, se vuelve tedioso distinguir entre la lógica empresarial (solo las fechas de las últimas dos semanas) y la lógica relacional (qué campos relacionan los clientes con los pedidos). El uso de la sintaxis JOIN con una cláusula ON hace que la lista WHERE sea más corta y hace que sea muy fácil ver cómo se relacionan las tablas entre sí.
- **Flexibilidad**. Digamos que necesitamos ver a todos los clientes incluso si no tienen un número de teléfono en el sistema. Con la segunda versión, es

fácil; simplemente cambie INNER JOIN phoneNumbers a LEFT JOIN phoneNumbers. Intente eso con la primera versión, y MySQL versión 5.0.12+ emitirá un error de sintaxis debido al cambio en la precedencia entre el operador de coma y la palabra clave JOIN. La solución es reorganizar la cláusula FROM o agregar paréntesis para anular la precedencia, y eso rápidamente se vuelve frustrante.

- **Portabilidad.** Los cambios en 5.0.12 se realizaron para alinearse con SQL: 2003. Si sus consultas utilizan una sintaxis estándar, le resultará más fácil cambiar a una base de datos diferente en caso de que surja la necesidad.

7. Subconsultas

Se trata de una técnica que permite utilizar el resultado de una consulta *SELECT* en otra consulta *SELECT*. Permite solucionar consultas que requieren para funcionar el resultado previo de otra consulta.

El estándar SQL define tres tipos de subconsultas:

- Subconsultas escalares. Son aquellas que devuelven una columna y una fila.
- Subconsultas de fila. Son aquellas que devuelven más de una columna, pero una única fila.
- Subconsultas de tabla. Son aquellas que devuelven una o varias columnas y cero o varias filas.

Se puede colocar el *SELECT* dentro de las cláusulas **WHERE**, **HAVING** o **FROM**. En este tema se estudiarán las subconsultas en la cláusula *WHERE*.

La sintaxis es:

```
SELECT listaExpresiones FROM tabla
WHERE expresión OPERADOR
(SELECT listaExpresiones
FROM tabla);
```

Los operadores/palabras reservadas que se pueden utilizar en las subconsultas son los siguientes:

- Operadores básicos de comparación: >, >=, <, <=, !=, <>, =.
- Predicados ALL y ANY.
- Predicado IN y NOT IN.
- Predicado EXISTS y NOT EXISTS.

Ejemplos: Seleccionar los empleados que cobran menos que GIL.

```
SELECT apellido, salario
FROM emple
WHERE salario <
(SELECT salario FROM emple
WHERE apellido='GIL');
```

Se pueden realizar esas subconsultas las veces que haga falta. Por ejemplo, si queremos saber los que cobran menos que GIL y más que SALA:

```
SELECT apellido, salario
FROM emple
WHERE salario <
    (SELECT salario FROM emple
     WHERE apellido='GIL')
AND salario >
    (SELECT salario FROM emple
     WHERE apellido='SALA');
```

En realidad, lo primero que hace la base de datos es calcular el resultado de los SELECT más internos, para luego calcular el más externo.

Las subconsultas siempre se deben encerrar entre paréntesis y se debería colocar a la derecha del operador relacional. Una subconsulta que utilice los valores >, <, >=, ... tiene que devolver un único valor, de otro modo ocurre un error.

Además, tienen que tener el mismo tipo de columna para relacionar la subconsulta con la consulta que la utiliza (no puede ocurrir que la subconsulta tenga dos columnas y ese resultado se compare usando una sola columna en la consulta general).

Pueden ponerse dos columnas en la condición y que la subconsulta devuelva también dos valores.

Ejemplo:

```
SELECT * FROM emple WHERE (oficio, dept_no) =
    (SELECT oficio, dept_no FROM emple WHERE apellido='REY');
```

Subconsultas con operadores básicos de comparación (>, >=, <, <=, !=, <>, =)

Los operadores básicos de comparación (>, >=, <, <=, !=, <>, =) se pueden usar cuando queremos comparar una expresión con el valor que devuelve una subconsulta.

Los operadores básicos de comparación los vamos a utilizar para realizar comparaciones con subconsultas que devuelven un único valor, es decir, una columna y una fila.

Ejemplo: Devuelve todos los productos que tienen un precio mayor o igual al producto más caro del fabricante Asus.

```
SELECT *
FROM producto
WHERE precio >= (SELECT MAX(precio)
                 FROM fabricante INNER JOIN producto
                 ON fabricante.codigo = producto.codigo_fabricante
                 WHERE fabricante.nombre = 'Asus');
```

El mismo ejemplo realizado completamente con subconsultas en vez de con INNER JOIN sería:

```
SELECT *
FROM producto
WHERE precio = (SELECT MAX(precio)
                FROM producto
                WHERE codigo_fabricante = (SELECT codigo
                                           FROM fabricante
                                           WHERE nombre = 'Asus'));
```

Subconsultas con ALL y ANY

ALL y ANY se utilizan con los operadores de comparación (>, >=, <, <=, !=, <>, =) y nos permiten comparar una expresión con el conjunto de valores que devuelve una subconsulta.

ALL y ANY los vamos a utilizar para realizar comparaciones con subconsultas que pueden devolver varios valores, es decir, una columna y varias filas.

La palabra reservada SOME es un alias de ANY, por lo que se puede usar indistintamente.

Ejemplo: Seleccionar el empleado que más cobra:

```
SELECT nombre, sueldo
FROM empleados
WHERE sueldo >= ALL (SELECT sueldo FROM empleados);
```

Ejemplo: De dos maneras diferentes, devuelve todos los productos de la base de datos que tienen un precio mayor o igual al producto más caro del fabricante Asus; una, haciendo uso de operadores de comparación y dos, haciendo uso de ALL.

-- Haciendo uso de operadores de comparación

```
SELECT *
FROM fabricante INNER JOIN producto
ON fabricante.codigo = producto.codigo_fabricante
WHERE precio >= (SELECT MAX(precio)
                 FROM fabricante INNER JOIN producto
                 ON fabricante.codigo = producto.codigo_fabricante
                 WHERE fabricante.nombre = 'Asus');
```

-- Haciendo uso del operador ALL

```
SELECT *
FROM fabricante INNER JOIN producto
ON fabricante.codigo = producto.codigo_fabricante
WHERE precio >= ALL (SELECT precio
                    FROM fabricante INNER JOIN producto
                    ON fabricante.codigo = producto.codigo_fabricante
                    WHERE fabricante.nombre = 'Asus');
```

Subconsultas con IN y NOT IN

IN y NOT IN nos permiten comprobar si un valor está o no incluido en un conjunto de valores, que puede ser el conjunto de valores que devuelve una subconsulta.

IN y *NOT IN* los vamos a utilizar para realizar comparaciones con subconsultas que pueden devolver varios valores, es decir, una columna y varias filas.

Ejemplo: Obtener los nombres de los empleados cuyos dni están en la tabla de directivos:

```
SELECT nombre
FROM empleados
WHERE dni IN (SELECT dni FROM directivos);
```

Ejemplo: Devuelve un listado de los clientes que no han realizado ningún pedido.

```
SELECT *
FROM cliente
WHERE id NOT IN (SELECT id_cliente FROM pedido);
```

Cuando se esté trabajando en subconsultas, *IN* y *= ANY* realizan la misma función. Por lo tanto, las siguientes consultas devolverían el mismo resultado:

```
SELECT s1 FROM t1 WHERE s1 = ANY (SELECT s1 FROM t2);
SELECT s1 FROM t1 WHERE s1 IN (SELECT s1 FROM t2);
```

Igual sucedería con *NOT IN* y *<> ALL*.

Si se necesita comprobar dos columnas en una consulta *IN*, se haría como en el ejemplo siguiente:

```
SELECT nombre
FROM empleados
WHERE (cod1, cod2) IN (SELECT cod1, cod2 FROM directivos);
```

Es importante tener presente que cuando hay un valor *NULL* en el resultado de la consulta interna, la consulta externa no devuelve ningún valor.

Ejemplo: Devuelve un listado con el nombre de los departamentos que no tienen empleados asociados.

```
SELECT nombre
FROM departamento
WHERE codigo NOT IN (SELECT codigo_departamento
                     FROM empleado);
```

Si la consulta interna *SELECT codigo_departamento FROM empleado*; devuelve algunas filas con valores *NULL*, entonces la consulta externa no devolverá ningún valor.

La forma de solucionarlo sería quitando los valores *NULL* de la consulta interna, tal y como se muestra a continuación:

```
SELECT nombre
FROM departamento
WHERE codigo NOT IN (SELECT codigo_departamento
                     FROM empleado
                     WHERE codigo_departamento IS NOT NULL);
```

Subconsultas con EXISTS y NOT EXISTS

Subconsultas que utilicen la palabra reservada *EXISTS* devolverán *true* si la subconsulta devuelve alguna fila; si no, devuelve *false*.

Ejemplo: Devuelve un listado de los clientes que no han realizado ningún pedido.

```
SELECT *  
FROM cliente  
WHERE NOT EXISTS (SELECT id_cliente  
                  FROM pedido  
                  WHERE cliente.id = pedido.id_cliente);
```

Se utiliza bastante en consultas correlacionadas.

Subconsultas correlacionadas

Son aquellas consultas en las que la consulta interna hace referencia a la consulta externa o, dicho de otro modo, ocurren cuando la subconsulta debe hacer referencia a una columna o varias de la consulta externa. Si las tablas usadas en la consulta y la subconsulta coinciden, habrá que usar alias de tabla.

Un caso general podría ser:

```
SELECT * FROM t1 WHERE column1 = ANY  
(SELECT column1 FROM t2 WHERE t2.column2 = t1.column2);
```

Entender este tipo de consultas puede resultar complicado. En este ejemplo, tenga en cuenta que la subconsulta contiene una referencia a una columna de *t1*, incluso aunque la cláusula *FROM* de la subconsulta no menciona una tabla *t1*. Por lo tanto, el SGBD busca fuera de la subconsulta y encuentra *t1* en la consulta externa.

Supongamos que la tabla *t1* contiene un registro en que *column1*=5 y *column2*=6; mientras, la tabla *t2* contiene un registro en que *column1*=5 y *column2*=7.

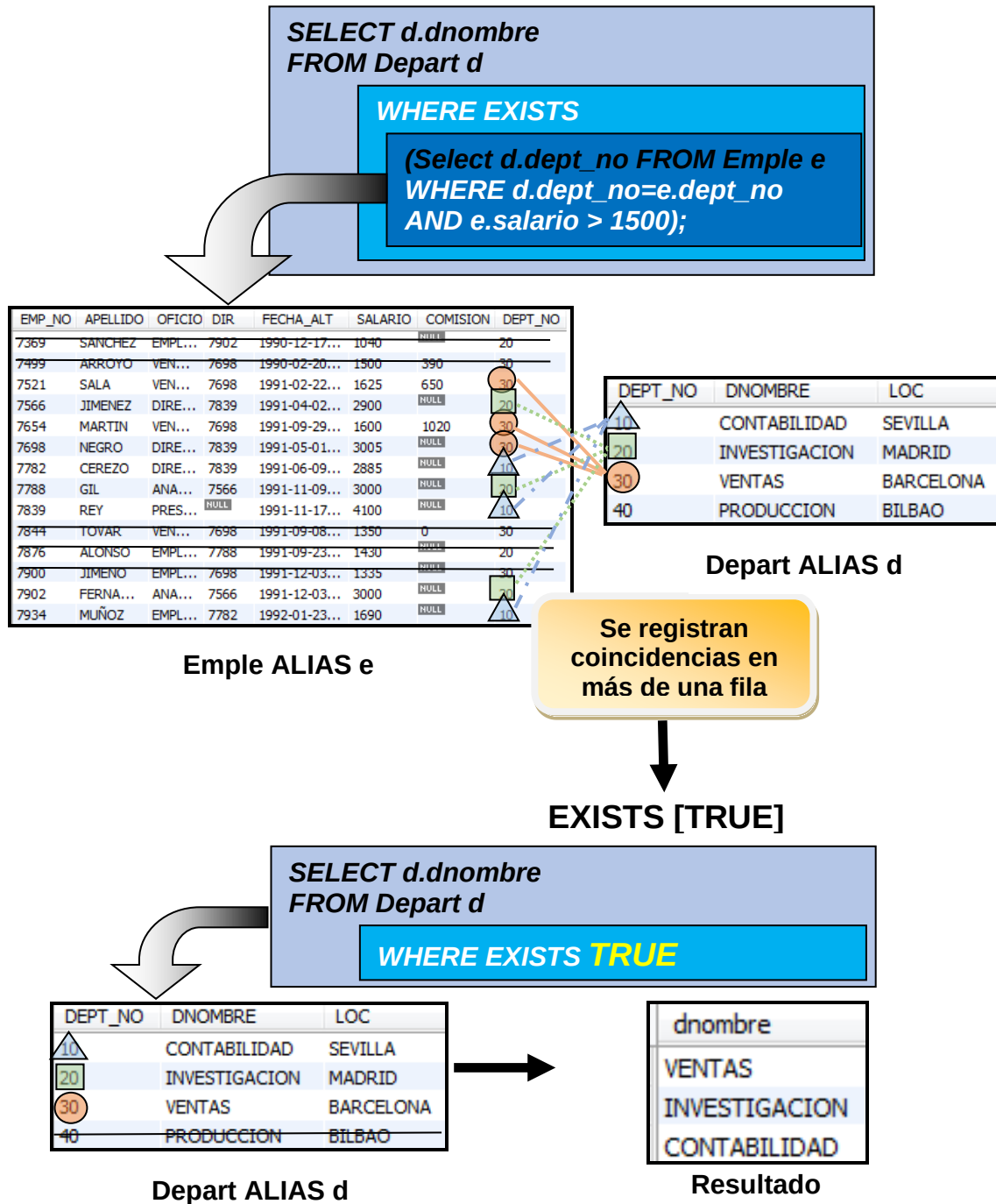
La expresión ...*WHERE column1 = ANY (SELECT column1 FROM t2)* sería *TRUE*, pero en este ejemplo, la cláusula *WHERE* dentro de la subconsulta es *FALSE* (ya que (5,6) no es igual a (5,7)), así que la subconsulta como un todo es *FALSE*.

Ejemplo: Departamentos en los que algún empleado gana más de 1500€

```
SELECT d.dnombre  
FROM Depart d WHERE EXISTS (Select d.dept_no  
                            FROM Emple e  
                            WHERE d.dept_no=e.dept_no  
                            AND e.salario > 1500);
```

dnombre
VENTAS
INVESTIGACION
CONTABILIDAD

Gráficamente:



Ejemplo: Mostrar los empleados cuyo salario sea el máximo de su departamento:

SELECT apellido, salario

FROM emple e WHERE salario >= ALL

*(SELECT salario FROM emple emp
WHERE e.dept_no=emp.dept_no);*

APELLIDO	SALARIO
NEGRO	3005
GIL	3000
REY	4100
FERNANDEZ	3000

Aclaración: Aunque hay sólo 3 departamentos con empleados (los nº 10, 20 y 30), se muestran 4 empleados porque en el departamento de Investigación (nº 20) el mayor sueldo (3000 €) lo tienen 2 empleados (Gil y Fernández).

Para más información sobre consultas correlacionadas con ejemplos explicados gráficamente se puede consultar:

<https://www.w3resource.com/sql/subqueries/corelated-subqueries-using-alias.php>

Subconsultas y JOINS

Como hemos visto en algún ejemplo, una subconsulta también se puede reescribir utilizando JOINS.

Casos generales, por ejemplo, una consulta con *IN* puede reescribirse utilizando la cláusula *DISTINCT*:

```
SELECT * FROM table1 WHERE col1 IN (SELECT col1 FROM table2);
```

Se convertiría en:

```
SELECT DISTINCT table1.* FROM table1, table2 WHERE table1.col1=table2.col1;
```

O, las siguientes consultas con *NOT IN* ó *NOT EXISTS*, que obtendrían el mismo resultado:

```
SELECT * FROM table1 WHERE col1 NOT IN (SELECT col1 FROM table2);
```

```
SELECT * FROM table1 WHERE NOT EXISTS (SELECT col1 FROM table2  
WHERE table1.col1=table2.col1);
```

También se podrían reescribir utilizando un *LEFT JOIN*, siendo a vez más eficiente esta última consulta:

```
SELECT table1.* FROM table1 LEFT JOIN table2 ON table1.id=table2.id WHERE  
table2.id IS NULL;
```

Hay algunos escenarios que requieren subconsultas en lugar de JOINS y otros al revés. Al ser un aspecto que excede el aprendizaje básico de este tema, se puede consultar más información en la documentación oficial de MariaDB: <https://mariadb.com/kb/en/subqueries-and-joins/>.

***** EJERCICIOS - Realizar los ejercicios 38-51 *****

***** EJERCICIOS DEL FINAL DEL TEMA – Realizar los ejercicios 52-111 *****

8. Anexos

a. Ejemplos de combinación de tablas

Sean las siguientes tablas:

Tabla ASIGNATURAS

ASIGNATURA	Aula	CODLIBRO
GBD	C5	1002
Sistemas O.	C8	221
Sistemas 2	C7	
Sistemas 1	C7	221
Programación	C1	100

Tabla LIBROS

CODLIBRO	NOMBRE	EDITORIAL
100	Programación Java	Mc Graw Hill
221	WINDOWS 7	Ra-ma
301	Análisis	Ra-ma
333	Certificación CISCO	CISCO
1002	Introducción SQL	Info Hoy

Veamos el resultado de las consultas siguientes:

*SELECT * FROM ASIGNATURAS, LIBROS;*

```
select * from asignaturas, libros;
```

Resultados Explicar Describir SQL Guardado Historial

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
GBD	C5	1002	100	Programación Java	Mc Graw Hill
GBD	C5	1002	221	WINDOWS 7	Ra-ma
GBD	C5	1002	301	Análisis	Ra-ma
GBD	C5	1002	333	Certificación CISCO	CISCO
GBD	C5	1002	1002	Introducción SQL	Info Hoy
Sistemas O.	C8	221	100	Programación Java	Mc Graw Hill
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	301	Análisis	Ra-ma
Sistemas O.	C8	221	333	Certificación CISCO	CISCO
Sistemas O.	C8	221	1002	Introducción SQL	Info Hoy
Sistemas 2	C7	-	100	Programación Java	Mc Graw Hill
Sistemas 2	C7	-	221	WINDOWS 7	Ra-ma
Sistemas 2	C7	-	301	Análisis	Ra-ma
Sistemas 2	C7	-	333	Certificación CISCO	CISCO
Sistemas 2	C7	-	1002	Introducción SQL	Info Hoy
Sistemas 1	C7	221	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas 1	C7	221	301	Análisis	Ra-ma
Sistemas 1	C7	221	333	Certificación CISCO	CISCO
Sistemas 1	C7	221	1002	Introducción SQL	Info Hoy
Programación	C1	100	100	Programación Java	Mc Graw Hill
Programación	C1	100	221	WINDOWS 7	Ra-ma
Programación	C1	100	301	Análisis	Ra-ma
Programación	C1	100	333	Certificación CISCO	CISCO
Programación	C1	100	1002	Introducción SQL	Info Hoy

25 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

*SELECT * FROM ASIGNATURAS CROSS JOIN LIBROS;*

```
select * from asignaturas cross join libros;
```

Resultados Explicar Describir SQL Guardado Historial

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
GBD	C5	1002	100	Programación Java	Mc Graw Hill
GBD	C5	1002	221	WINDOWS 7	Ra-ma
GBD	C5	1002	301	Análisis	Ra-ma
GBD	C5	1002	333	Certificación CISCO	CISCO
GBD	C5	1002	1002	Introducción SQL	Info Hoy
Sistemas O.	C8	221	100	Programación Java	Mc Graw Hill
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	301	Análisis	Ra-ma
Sistemas O.	C8	221	333	Certificación CISCO	CISCO
Sistemas O.	C8	221	1002	Introducción SQL	Info Hoy
Sistemas 2	C7	-	100	Programación Java	Mc Graw Hill
Sistemas 2	C7	-	221	WINDOWS 7	Ra-ma
Sistemas 2	C7	-	301	Análisis	Ra-ma
Sistemas 2	C7	-	333	Certificación CISCO	CISCO
Sistemas 2	C7	-	1002	Introducción SQL	Info Hoy
Sistemas 1	C7	221	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas 1	C7	221	301	Análisis	Ra-ma
Sistemas 1	C7	221	333	Certificación CISCO	CISCO
Sistemas 1	C7	221	1002	Introducción SQL	Info Hoy
Programación	C1	100	100	Programación Java	Mc Graw Hill
Programación	C1	100	221	WINDOWS 7	Ra-ma
Programación	C1	100	301	Análisis	Ra-ma
Programación	C1	100	333	Certificación CISCO	CISCO
Programación	C1	100	1002	Introducción SQL	Info Hoy

25 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

SELECT *
FROM ASIGNATURAS A, LIBROS L
WHERE A.CODLIBRO=L.CODLIBRO;

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
Programación	C1	100	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
GBD	C5	1002	1002	Introducción SQL	Info Hoy

4 filas devueltas en 0,02 segundos

[Exportación de CSV](#)

SELECT * FROM ASIGNATURAS
NATURAL JOIN LIBROS;

CODLIBRO	ASIGNATURA	AULA	NOMBRE	EDITORIAL
100	Programación	C1	Programación Java	Mc Graw Hill
221	Sistemas 1	C7	WINDOWS 7	Ra-ma
221	Sistemas O.	C8	WINDOWS 7	Ra-ma
1002	GBD	C5	Introducción SQL	Info Hoy

4 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

SELECT * FROM ASIGNATURAS
INNER JOIN LIBROS USING (CODLIBRO);

CODLIBRO	ASIGNATURA	AULA	NOMBRE	EDITORIAL
100	Programación	C1	Programación Java	Mc Graw Hill
221	Sistemas 1	C7	WINDOWS 7	Ra-ma
221	Sistemas O.	C8	WINDOWS 7	Ra-ma
1002	GBD	C5	Introducción SQL	Info Hoy

4 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

SELECT * FROM ASIGNATURAS
JOIN LIBROS ON
ASIGNATURAS.CODLIBRO=LIBROS.CODLIBRO;

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
Programación	C1	100	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
GBD	C5	1002	1002	Introducción SQL	Info Hoy

4 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

SELECT * FROM ASIGNATURAS
LEFT OUTER JOIN LIBROS
ON ASIGNATURAS.CODLIBRO=LIBROS.CODLIBRO;
 Salen todas las asignaturas

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
Programación	C1	100	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
GBD	C5	1002	1002	Introducción SQL	Info Hoy
Sistemas 2	C7	-	-	-	-

5 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

```
SELECT * FROM ASIGNATURAS
RIGHT OUTER JOIN LIBROS
ON ASIGNATURAS.CODLIBRO=LIBROS.CODLIBRO;
```

Salen todos los libros

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
GBD	C5	1002	1002	Introducción SQL	Info Hoy
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Programación	C1	100	100	Programación Java	Mc Graw Hill
-	-	-	333	Certificación CISCO	CISCO
-	-	-	301	Análisis	Ra-ma

```
SELECT * FROM ASIGNATURAS
FULL OUTER JOIN LIBROS
ON ASIGNATURAS.CODLIBRO=LIBROS.CODLIBRO;
```

Salen todos los libros y las asignaturas.

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
Programación	C1	100	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
GBD	C5	1002	1002	Introducción SQL	Info Hoy
Sistemas 2	C7	-	-	-	-
-	-	-	333	Certificación CISCO	CISCO
-	-	-	301	Análisis	Ra-ma

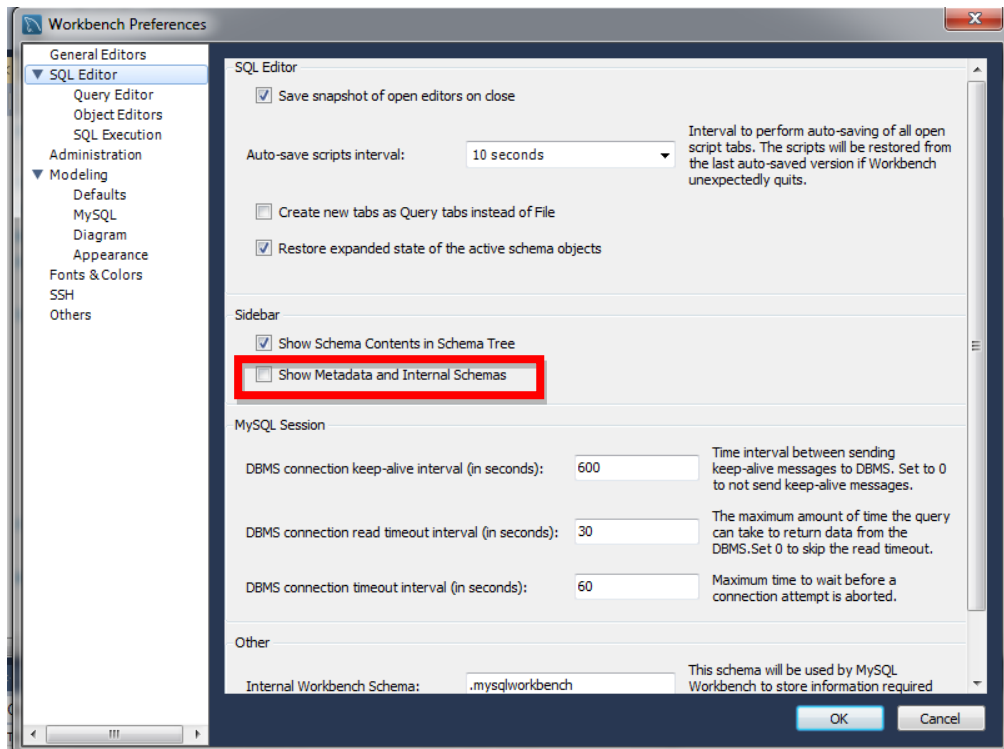
7 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

b. Diccionario de datos

Todas las bases de datos disponen de posibilidades para consultar el diccionario de datos. Siguiendo las reglas de Codd, la forma de consultar los metadatos es la misma que en el resto de tablas. Es decir, existen tablas que, en lugar de contener datos, contienen los metadatos. En el caso de SQL estándar, el diccionario de datos es accesible mediante el esquema de información (INFORMATION_SCHEMA), un esquema especial que contiene el conjunto de vistas con el que se pueden consultar los metadatos de la base de datos.

Para ver las bases de datos del sistema desde Mysql WorkBench debemos cambiar una opción de la configuración, accesible desde el menú Edit – Preferences. Allí, seleccionamos la opción SQL Editor y luego se marca la casilla correspondiente a “Show Metadata and Internal Schemas”, como se puede ver en la siguiente imagen.



Una vez hecho esto, si actualizamos las bases de datos de la sección Schemas, podremos ver que aparece “information_schema”.

Una vez situados en dicho esquema, podemos realizar numerosas consultas, como por ejemplo, conocer los nombres de las tablas creadas en una base de datos, como se puede ver a continuación:

```
SELECT table_name, table_type, engine
FROM information_schema.tables
WHERE table_schema = 'instituto'
ORDER BY table_name DESC;
```

También podemos obtener las columnas de una tabla o vista de una base de datos con la orden:

```
SELECT
    t1.COLUMN_NAME AS columna_nombre,
    t1.COLUMN_DEFAULT AS columna_defecto,
    t1.IS_NULLABLE AS columna_nulo,
    t1.DATA_TYPE AS columna_tipo_dato,
    IFNULL(t1.NUMERIC_PRECISION,
    t1.CHARACTER_MAXIMUM_LENGTH) AS columna_longitud,
    t1.COLUMN_COMMENT AS columna_descripcion
FROM
    INFORMATION_SCHEMA.COLUMNS t1
WHERE
    t1.TABLE_SCHEMA = 'instituto' AND
    t1.TABLE_NAME = 'alumno'
ORDER BY
    t1.ORDINAL_POSITION;
```

O también obtener las restricciones de las columnas de una tabla de una base de datos mediante la consulta:

```
SELECT
    t1.CONSTRAINT_TYPE AS tipo_restriccion,
    t2.COLUMN_NAME columna_nombre,
    t2.REFERENCED_TABLE_NAME tabla_referencia,
    t2.REFERENCED_COLUMN_NAME columna_referencia
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS t1
LEFT JOIN INFORMATION_SCHEMA.KEY_COLUMN_USAGE t2
ON t1.CONSTRAINT_CATALOG = t2.CONSTRAINT_CATALOG
AND t1.CONSTRAINT_SCHEMA = t2.CONSTRAINT_SCHEMA
AND t1.CONSTRAINT_NAME = t2.CONSTRAINT_NAME
WHERE
    t1.TABLE_SCHEMA = 'instituto' AND
    t1.TABLE_NAME = 'alumno' AND
    t2.COLUMN_NAME = 'id'
ORDER BY
    t1.CONSTRAINT_TYPE DESC;
```

Se pueden generar muchas más consultas para obtener la información que se desee. De hecho, es importante saber acceder a la información del diccionario de datos para extraer información de la base de datos cuando no se disponga de un software intermediario y se deba trabajar directamente en línea de comandos.

Información extra de las tablas de “Information Schema” se puede consultar en:

<https://mariadb.com/kb/en/information-schema-tables/>

c. Tablas de los ejercicios

EMPLE

EMP_NO	APELLIDO	OFICIO	DIR	FECHA_ALT	SALARIO	COMISION	DEPT_NO
7369	SANCHEZ	EMPL...	7902	1990-12-17...	1040	NULL	20
7499	ARROYO	VEN...	7698	1990-02-20...	1500	390	30
7521	SALA	VEN...	7698	1991-02-22...	1625	650	30
7566	JIMENEZ	DIRE...	7839	1991-04-02...	2900	NULL	20
7654	MARTIN	VEN...	7698	1991-09-29...	1600	1020	30
7698	NEGRO	DIRE...	7839	1991-05-01...	3005	NULL	30
7782	CEREZO	DIRE...	7839	1991-06-09...	2885	NULL	10
7788	GIL	ANA...	7566	1991-11-09...	3000	NULL	20
7839	REY	PRES...	NULL	1991-11-17...	4100	NULL	10
7844	TOVAR	VEN...	7698	1991-09-08...	1350	0	30
7876	ALONSO	EMPL...	7788	1991-09-23...	1430	NULL	20
7900	JIMENO	EMPL...	7698	1991-12-03...	1335	NULL	30
7902	FERNA...	ANA...	7566	1991-12-03...	3000	NULL	20
7934	MUÑOZ	EMPL...	7782	1992-01-23...	1690	NULL	10

DEPART

DEPT_NO	DNOMBRE	LOC
10	CONTABILIDAD	SEVILLA
20	INVESTIGACION	MADRID
30	VENTAS	BARCELONA
40	PRODUCCION	BILBAO

ALUM1213

DNI	NOMBRE	APELLIDOS	FECHA_NAC	DIRECCION	POBLACION	PROVINCIA	CURSO	NIVEL	CLASE	FALTAS1	FALTAS2	FALTAS3
16432432J	Ana	Martínez Escobés	1994-04-03 00:00:00	C/ Madrid, 12	Lardero	La Rioja	1	ASIR	V	3	8	1
16562562W	Eduardo	Bravo López	1989-02-02 00:00:00	C/ Clavijo, 40	Logroño	La Rioja	2	DAM	D	5	2	3
16654654A	Luis Javier	Martínez Lajo	1993-03-03 00:00:00	C/ Bretón, 14	Logroño	La Rioja	2	DAM	D	1	0	1
16789789A	Eduardo	Castor Flores	1989-10-08 00:00:00	C/ Los almendros, 4	Villamediana	La Rioja	1	DAM	D	0	0	0
34448974N	CONCHA	ESTRUCH VIDAL	1985-11-10 00:00:00	C/ MEDICO FELIX, 12	BERROCALEJO	CACERES	2	DAM	D	0	0	0
34448999N	DORA	MARTIN BREÑA	1994-11-10 00:00:00	C/ MEDICO FELIX, 10	CACERES	CACERES	1	ASIR	D	0	0	0

ALUMNOS

DNI	APENOM	DIREC	POBLA	TELEF
16344345	Alcalde García, Elena	C/Las Matas, 24	Madrid	917766545
16448242	Cerrato Vela, Luis	C/Mina 28 - 3A	Madrid	916566545
16882942	Díaz Fernández, María	C/Luis Vives 25	Móstoles	915577545

ASIGNATURAS

COD	NOMBRE
1	Programación
2	Sist. Informáticos
3	Bases de datos
4	FOL
5	Inglés técnico
6	Lenguajes de marcas
7	Seguridad
8	Entornos

NOTAS

DNI	COD	NOTA
16344345	1	6
16344345	2	5
16344345	3	6
16344345	4	5
16344345	5	6
16344345	6	8
16344345	7	4
16448242	4	6
16448242	5	8
16448242	6	4
16448242	7	5
16882942	4	8
16882942	5	7
16882942	6	8
16882942	7	9

NOTAS_ALUMNOS

NOMBRE_ALUMNO	NOTA1	NOTA2	NOTA3
Alcalde García, M. Luisa	5	5	5
Benito Martín, Luis	7	6	8
Casas Martínez, Manuel	7	5	5
Casas Sánchez, Ana	6	9	8
Díaz Sánchez, Maria	NULL	NULL	7
Martín Salas, Fernando	6	5	7
Sánchez Fuentes, Susana	4	5	4
Jover Sánchez, Mario	7	7	5
Lafuente Sánchez, Sara	6	6	6
Sánchez Lahera, Fernando	3	3	6

LIBRERÍA

TEMA	ESTANTE	EJEMPLARES
INFORMATICA	A	15
ECONOMIA	A	10
DEPORTES	B	8
FILOSOFIA	C	7
DIBUJO	C	10
MEDICINA	C	16
BIOLOGIA	A	11
GEOLOGIA	D	7
SOCIEDAD	D	9
LABORES	B	20
JARDINERIA	E	6

LECTOR

NUMLECTOR	NOMLECTOR	TIPOLECTOR
1	Posadas Gil, Inés	3
2	Sánchez Pons, José	3
3	Gómez Sáez, Miguel	3
10	Santana Páez, Eva	4
11	Betancor Díaz, Yolanda	4
20	Blasco Ulla, Juan Luis	1
22	Marín Gómez, Ana	1
30	Plaza Ortiz, Juan	8

LIBRO

CODLIBRO	TITULO	NUMPAG	EDITORIAL	AUTOR
12	Don Quijote I	517	Anaya	Cervantes
13	Don Quijote II	611	Anaya	Cervantes
20	El principito	120	Andina	Saint_Exupery
35	Fortunata y Jacinta	625	PlazaJanes	Pérez-Galdós
40	Caperucuta en Manhattan	310	Salamandra	Martín GAite
41	La piel del tambor	280	Circulo	Pérez-Reverte
42	Afrodita	421	Espasa	Isabel Allende
50	Viajar	675	BBVA	NULL
51	España en sus caminos	534	BBVA	NULL
52	Aeroguía del litoral	340	Planeta	NULL
65	Don Quijote I	578	Planeta	Cervantes
66	Don Quijote II	600	Planeta	Cervantes

PRÉSTAMO

CODLIBRO	NUMLECTOR	FPRESTA	DIAS
12	20	2012-04-12 00:00:00	54
13	11	2012-05-10 00:00:00	26
20	11	2012-06-01 00:00:00	4
35	1	2012-04-15 00:00:00	51
40	3	2011-11-01 00:00:00	217
42	30	2012-05-05 00:00:00	31
50	20	2012-03-05 00:00:00	92
51	20	2012-05-06 00:00:00	30
52	22	2012-05-06 00:00:00	30